COMP2300 Set 6 Practice Exam Solutions

Part A: Multiple Choice

A1. B

A2. C

A3. A

A4. C

A5. B

A6. C

A7. B

A8. C

A9. C

A10. B

Part B: Computational Problems

B1. Branch target address

(a)

$$0x000005 \ll 2 = 0x000014$$

(b) Since the value is positive, sign extension leaves it unchanged:

$$\texttt{ExtImm} = 0x00000014$$

(c) ARM branch rule:

$$\texttt{BTA} = (\texttt{PC} + 8) + \texttt{ExtImm}$$

Hence:

$$\texttt{PC} + 8 = 0x2048$$

$$\texttt{BTA} = 0x2048 + 0x14 = 0x205C$$

B2. Register roles in a call

(a) `LR` holds the return address after `BL foo`.

(b) In the lecture convention, `R0` and `R1` are caller-saved / non-preserved argument-temporary registers.

(c) The caller must save them before the call, for example by copying them elsewhere or pushing them on the stack.

B3. Stack-pointer updates

(a) Three 32-bit registers are pushed:

$$\texttt{SP} = 0x1000 - 12 = 0x0FF4$$

(b) Two 32-bit locals require 8 more bytes:

$$\texttt{SP} = 0x0FF4 - 8 = 0x0FEC$$

(c) Deallocate locals:

$$0x0FEC + 8 = 0x0FF4$$

Then pop 3 registers:

$$0x0FF4 + 12 = 0x1000$$

Final `SP` is `0x1000`.

B4. Prologue and epilogue reasoning

(a) Because the function makes a nested call, a new `BL` will overwrite `LR`. If the current function wants to return correctly, it must preserve the old `LR`.

(b) `R4` and `R7` are callee-saved in the course convention, so if the function modifies them, it must save and later restore them.

(c) One possible answer is:

```
PUSH {R4, R7, LR}
...
POP  {R4, R7, LR}
MOV  PC, LR
```

B5. Stack frames in recursion

(a) Active calls for $n = 3, 2, 1$ means 3 active stack frames.

(b) Each call pushes two 32-bit registers:

$$2 \times 4 = 8 \text{ bytes per frame}$$

Total:

$$3 \times 8 = 24 \text{ bytes}$$

(c)

$$\texttt{SP} = 0x8000 - 24 = 0x7FE8$$

B6. Caller-save vs callee-save

(a) `main` is responsible for preserving the old value of `R2`, because `R2` is a caller-saved register in the lecture convention.

(b) `f` is responsible for preserving `R4` and `R5` if it modifies them, because they are callee-saved.

(c) When `f` calls `g`, the new `BL` to `g` will overwrite `LR`. Therefore `f` must save its own return address first.

(d) It is more efficient because only registers that matter to the current caller/callee interaction are saved. Many calls do not use every register, so unconditional full save/restore wastes time and stack space.

Part C: Past Exam Questions

C1. Functions with stack support

(a) A link register stores the return address without forcing every call to access memory immediately.

(b) Stack push/pop support provides a structured way to save registers, locals, arguments beyond register capacity, and return information.

(c) A branch-and-link instruction combines control transfer with return-address capture, which is exactly what function calls need.

C2. Calling convention design

One valid design:

- `r0-r1`: caller-saved / argument / return / temporaries
- `r2-r3`: callee-saved

Justification:

- Fast-changing temporaries should be caller-saved because many small functions use them briefly.
- Longer-lived values benefit from callee-saved behavior so callers do not have to save them repeatedly.

Any convention with a clear, internally consistent contract earns full credit.

C3. Returning without a link register

If there is no dedicated link register, the caller (or callee prologue) must save the return address explicitly on the stack at call time. The called function then restores that saved address before returning and transfers control back to it, for example by loading it into the program counter or by branching indirectly through a general-purpose register holding the saved address.